CS 486/686
Training Models From Scratch

Yuntian Deng

Lecture 17

Core loop: data → model → loss → gradient → update

Recorded lecture (asynchronous)

> 🎥 This lecture is **pre-recorded** - I'm away at **ICML** this week, so there's no in-person class today. Watch at your own pace.
>
> 📅 **Due today (Tue Jul 7):** Chat 8 and the CS686 project proposal.
>
> 💬 Questions? Post on **Piazza** - the TAs are available, and I'll follow up when I'm back.

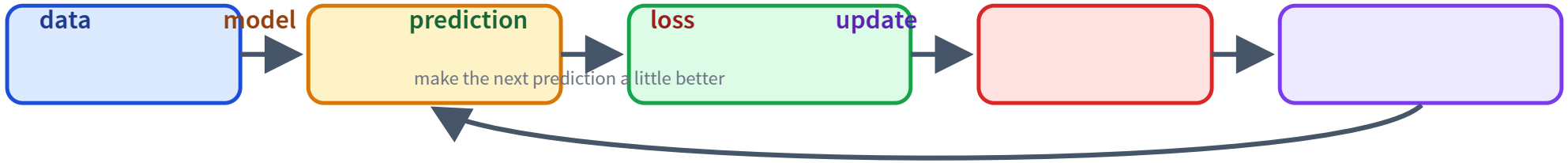
Today's target

By the end, this loop should feel readable:

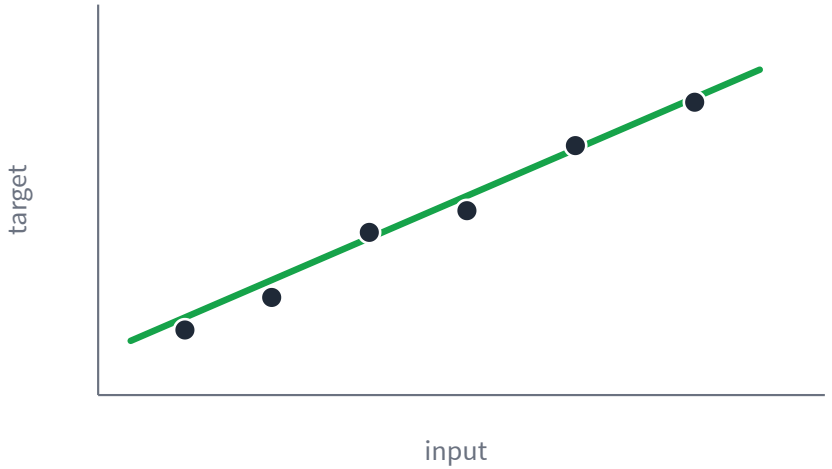
```
for x, y in loader:
    pred = model(x)
    loss = loss_fn(pred, y)
    loss.backward()
    optimizer.step()
    optimizer.zero_grad()
```

Everything later - neural nets, LLMs, diffusion - scales up this idea.

The learning recipe



Parameters are the knobs

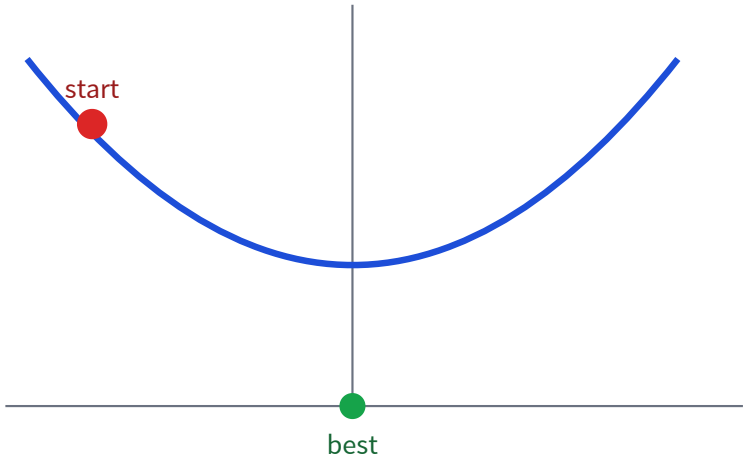


The data is fixed.

The model has tunable **parameters**: w and b.

Training means choosing parameters that make predictions good.

Start with one knob



Forget ML for one slide.

We only need to make a loss small.

This is the smallest possible version of training.

Move downhill

The gradient points uphill.

$$x \leftarrow x - \eta \frac{dL}{dx}$$

$-dL/dx$

direction that decreases loss

η

learning rate: how big a step?

PyTorch computes gradients for us

Manual derivative

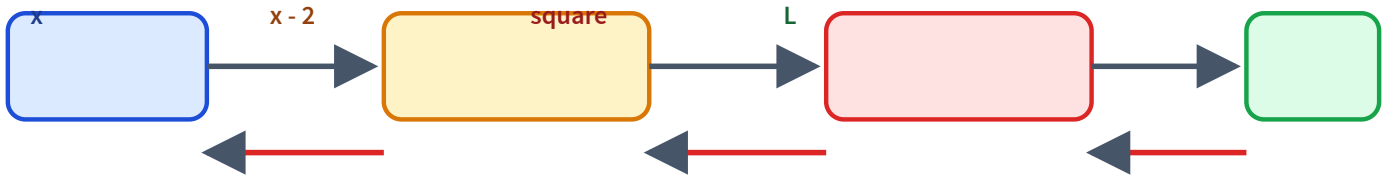
```
def loss(x):
    return (x - 2) ** 2

def manual_grad(x):
    return 2 * (x - 2)
```

Automatic differentiation

```
x = nn.Parameter(torch.tensor(-4.0))
L = (x - 2) ** 2
L.backward()
print(x.grad)  # -12
```

For millions of parameters, we rely on autograd.

Autograd follows the computation graph



backward pass: chain rule in reverse

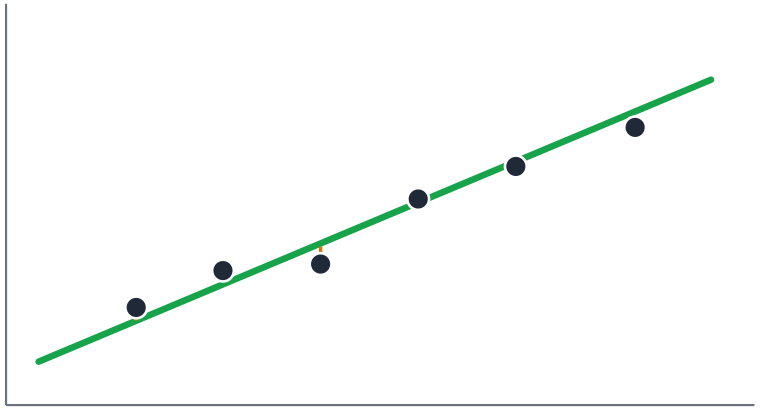
From one knob to many

A model has many parameters, collected as θ.

$$\theta \leftarrow \theta - \eta \nabla_\theta L(\theta)$$

> `model.parameters()` = θ
>
> `loss.backward()` = compute $\nabla_\theta L$
>
> `optimizer.step()` = update θ

Example 1: regression



Prediction is a number.

$$L = \frac{1}{m}\sum_i (\hat{y}_i - y_i)^2$$

Squared error punishes large residuals.

Regression in PyTorch

```
model = nn.Linear(1, 1)
loss_fn = nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)

pred = model(x)
loss = loss_fn(pred, y)
loss.backward()
optimizer.step()
optimizer.zero_grad()
```

Same recipe, now the knob is a line.

Example 2: classification

Now the target is a class, not a number.

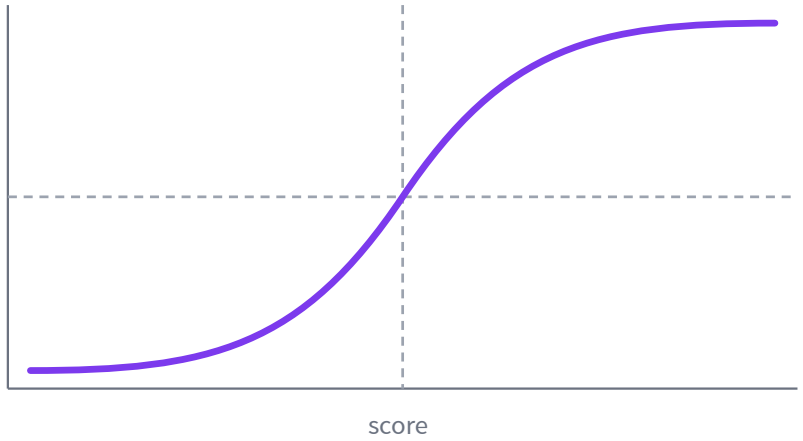
Regression

$\hat{y} = 217.3$

Classification

$P(\text{spam} \mid x) = 0.91$

Logistic regression



Compute a score:

$$z = w^\top x + b$$

Squash it into a probability:

$$\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$$

Loss for classification

Punish being confidently wrong.

$$-y \log \hat{y} - (1 - y) \log(1 - \hat{y})$$

true: spam

$\hat{y} = 0.99$ → small loss

true: spam

$\hat{y} = 0.01$ → huge loss

Many classes: softmax

One score per class; softmax turns scores into probabilities.

$$P(y = k \mid x) = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

Next-token prediction is this, but over a vocabulary.

The full loop, annotated

```
for x, y in loader:                  # mini-batch
    pred = model(x)                  # forward pass
    loss = loss_fn(pred, y)          # measure error
    loss.backward()                  # compute gradients
    optimizer.step()                 # update parameters
    optimizer.zero_grad()            # reset gradient buffers
```

This is the code pattern to recognize for the rest of the module.

Why mini-batches?

one epoch = one pass over the dataset

Mini-batches make updates cheap, stable enough, and GPU-friendly.

Evaluation is not training

train

updates weights

validation

chooses settings

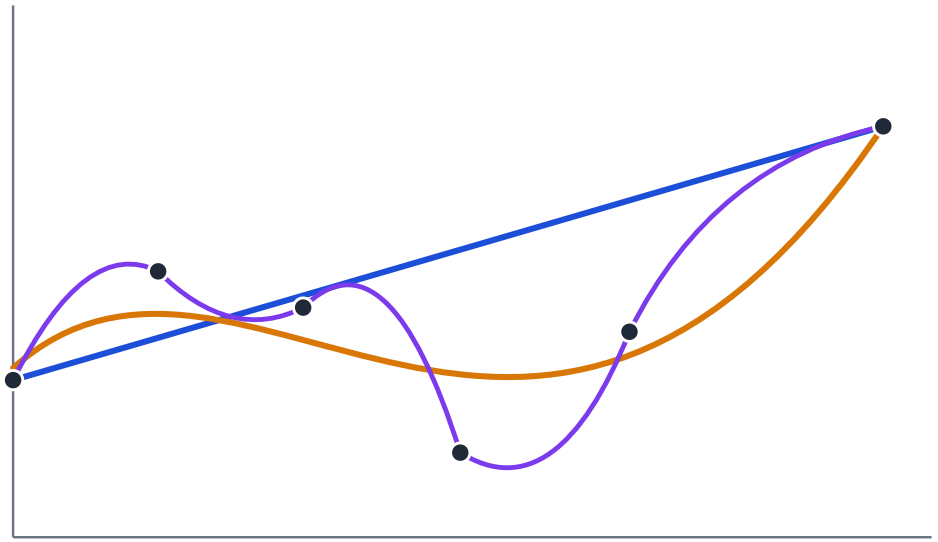
test

final estimate

Do not tune on the test set.

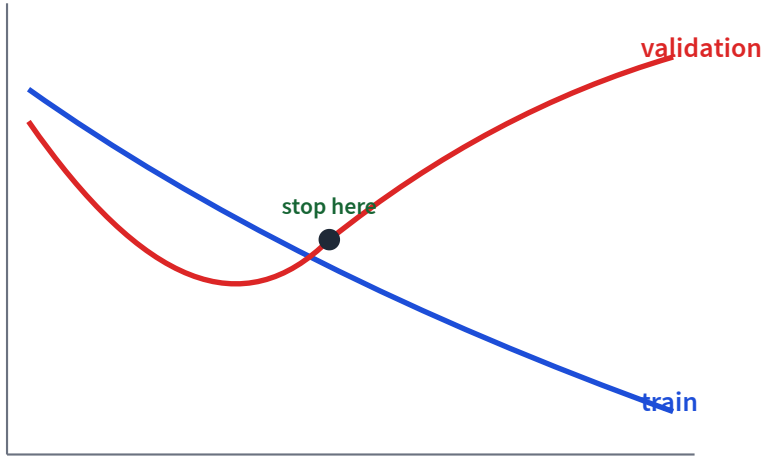
More power is not always better

Same data, increasingly flexible models.



underfit good fit overfit

Watch train and validation loss



Training loss can keep improving.

Validation loss tells us when generalization is getting worse.

Three ways to fight overfitting

More data

show the model more of the world

Regularize

prefer simpler weights

Early stop

stop when validation turns up

Example: $L(\theta) + \lambda ||\theta||^2$

What carries forward?

Later models change the architecture and loss.

neural net

more layers

LLM

predict next token

diffusion

predict noise

The training loop stays recognizable.

Recap - next: learned representations

✓ A model learns by minimizing a **loss**.

✓ **Autograd** computes gradients through a graph.

✓ Regression uses squared error; classification uses **cross-entropy**.

✓ Validation loss catches **overfitting**.

L18: if linear models use fixed features, how do neural nets learn their own?